

Contents

SYSTEM DESIGN CASE STUDY	2
Introduction & Executive Context	3
Executive Summary	4
The Problem Statement	4
Target State Architecture & Structural Design	5
Comprehensive Solution Summary	7
End-to-End Customer Journey.....	10
Transaction Data-Flows	13
Security Architecture.....	15
Disaster Recovery (DR) Plan	17
Architecture Tradeoffs.....	20
Architecture Decision Records	22
AWS Well-Architected Framework Review	25
Architecture Risks.....	28
Architectural Recommendations.....	32
Boundary Conditions for Reference Architecture	35
Appendix A: Non-Functional Requirements (NFR) Matrix	39
Appendix B: Cost Optimization & Attribution Matrix	42
Appendix C: Enterprise Security Risk Register	45
Appendix D: Enterprise RACI Matrix	48
About the Author	50

SYSTEM DESIGN CASE STUDY

OpenBanking Reference Architecture on AWS

Prepared by:

Amit Kulkarni

Introduction & Executive Context

The architecture diagram illustrates a robust, cloud-native reference model for Open Banking, specifically designed using AWS Well-Architected principles to support secure financial ecosystems. At its core, the framework establishes a secure, standardized mechanism for data exchange between consumers, licensed third parties, and trust service providers using mutual TLS (mTLS) verification. Designed for highly regulated banking environments, it bridges traditional infrastructure—such as on-premises data centers running Kafka, MQ, and core payment systems—with a modern, agile AWS cloud environment.

The architecture optimizes security and regulatory compliance by implementing a multi-layered defense strategy, isolating edge routing in a dedicated Networking Account while confining core business logic within distinct Open Banking API Accounts. This multi-account governance structure separates responsibilities effectively, utilizing AWS Shield, Web Application Firewall (WAF), and AWS API Gateways to throttle and protect traffic at the perimeter. Downstream services rely on decoupled microservices to manage Account Information Services, Payments Services, and automated webhooks, backed by replicated, synchronized storage layer engines like AWS RDS. Enterprise identity federation is seamlessly achieved via a dedicated Identity Provider account that integrates with leading standards like ForgeRock, Ping Identity, and Keycloak. Furthermore, the architecture guarantees operational excellence and strict compliance auditing through a comprehensive suite of centralized monitoring tools, including AWS CloudTrail, AWS CloudWatch, and AWS Config. Ultimately, this scalable blueprint empowers financial institutions to safely open their APIs, drive innovation, and remain compliant with shifting global open banking mandates.

.

Executive Summary

This architecture delivers a highly secure, resilient, and compliant AWS reference blueprint designed specifically for Open Banking integration. By seamlessly bridging legacy on-premises data centers with advanced cloud-native services, the solution enables financial institutions to securely expose transactional and payment APIs to authorized third-party providers. The design prioritizes zero-trust security architecture, mandating mutual *TLS* (mTLS) for all external connections and enforcing strict multi-account isolation across networking, identity management, and core API microservices. Supported by robust perimeter defense tools and an automated, centralized logging and monitoring framework, this infrastructure mitigates modern cybersecurity risks while satisfying strict global financial regulations. Ultimately, this scalable foundation accelerates digital transformation, allowing banks to foster third-party innovation, scale customer digital services, and ensure continuous operational excellence without compromising data sovereignty.

The Problem Statement

Modern financial institutions struggle to securely share financial data and open their legacy ecosystems to third-party providers while maintaining regulatory compliance. Traditional banking frameworks are often bound to rigid, monolithic on-premises infrastructure that cannot dynamically scale or safely expose APIs at the edge.

This technological gap creates several critical challenges:

1. **Inflexible Legacy Systems and Data Silos:** Core banking data is locked inside on-premises data centers, legacy mainframes, *Kafka* pipelines, and traditional *message queues* (MQ). These systems lack the agility and native web capabilities required to support modern, high-velocity API ecosystems.
2. **Complex and Evolving Regulatory Mandates:** Strict global open banking frameworks demand secure data exchange and explicit customer consent management. Implementing these rigorous, localized mandates on top of older infrastructure is complex, costly, and error-prone.
3. **Expanded Attack Surface and Security Vulnerabilities:** Opening financial rails to external consumers and third-party applications vastly increases exposure to malicious actors. Without advanced perimeter defense and

zero-trust isolation, institutions risk catastrophic data breaches, API abuse, and distributed denial-of-service (*DDoS*) attacks.

4. **Fragmented Identity and Consent Management:** Banks lack a unified, scalable system to federate customer identities and track explicit consent permissions across diverse third-party applications. This makes it difficult to securely authenticate users without compromising sensitive credentials.
5. **Operational Blind Spots and Audit Vulnerabilities:** Tracking end-to-end data flows between external API consumers and internal banking registries is nearly impossible without automated, centralized monitoring. This lack of visibility leaves institutions vulnerable during compliance audits and slows down incident response.

Target State Architecture & Structural Design

System Architecture Overview

The target state architecture establishes a highly secure, scalable, multi-tier cloud topology engineered for modern *Open Banking API* Modernization, aligning with the official *AWS Well-Architected* reference standard. The platform moves away from rigid legacy constraints by decoupling public api entry endpoints, third-party authentication authorities, and core financial domains from containerized compute workloads. All transactions are securely routed through isolated, multi-account enterprise *Virtual Private Clouds* (VPCs), executing across highly available cloud structures backed by low-latency integration infrastructure, secure third-party sandboxes, and cloud-native databases.

Core Functional Processing Tiers

1. Perimeter Defense and Perimeter Ingress

The outer layer utilizes a dedicated, highly protected environment to intercept, inspect, and route all incoming traffic from external entities. This ensures that only validated requests penetrate the internal network boundaries.

1. **Edge Infrastructure and Traffic Control:** *AWS Route 53* and *AWS CloudFront* handle global DNS resolution and content delivery, optimizing latency for third-party consumers.

2. **Distributed Denial of Service Protection:** *AWS Shield* provides specialized perimeter defense against large-scale network and transport layer attacks.
3. **Advanced Web Application Filtering:** The *AWS Web Application Firewall* (WAF) inspects incoming requests at the application layer to block common web exploits and custom injection threats.
4. **Mutual Transport Layer Authentication:** Secure edge routing requires mutual *TLS (mTLS)* to strictly verify the identity of licensed third parties and trust providers.

2. Network Isolation and Multi-Account Governance

The architecture separates concerns across multiple *AWS accounts* and *virtual private clouds* (VPCs) to enforce security boundaries and minimize blast radiuses.

1. **Dedicated Networking Account:** Isolates edge components, public-facing load balancers, and external *AWS API gateways* away from the core business registries.
2. **Open Banking API Account:** Confines the sensitive microservices, data processors, and storage engines to a distinct, highly restricted cloud partition.
3. **Transit Gateway Interconnect:** An *AWS Transit Gateway* acts as a central hub, managing secure, high-speed routing between different *AWS accounts* and on-premises systems.
4. **Private Integration Links:** *AWS PrivateLink*, *Network Load Balancers* (NLBs), and *VPC Endpoints* establish secure connections to identity and developer sandbox accounts without traversing the public internet.

3. Core Open Banking Application and Data Layers

This layer manages consumer transactions, consent permissions, and financial service logic via highly decoupled backend components.

1. **Specialized Banking Microservices:** Segregates application logic into dedicated blocks for Account Information Services, Payments Services, and real-time Webhooks.
2. **Centralized Identity Federation:** A dedicated Identity Provider account natively supports third-party engines like *ForgeRock*, *Ping Identity*, and *Keycloak* for robust user token management.

3. **Synchronized System of Records:** *AWS RDS* operates as the primary, relational database engine, remaining actively synchronized with underlying physical on-premises databases.
4. **Consent and Metrics Storage:** Specialized sub-systems catalog explicit consumer consent permissions, aggregated product data, and real-time API performance metrics.

4. Legacy Integration and Operational Governance

The blueprint bridges stateful cloud resources with existing data centers while maintaining continuous compliance monitoring.

1. **Hybrid Connectivity Channels:** *AWS Direct Connect* and *AWS VPN* establish redundant, private communication lines back to the Bank's physical *Data Center*.
2. **On-Premises Event Streaming:** The hybrid network links cloud storage to physical core payment systems, active mainframes, *Apache Kafka* clusters, and *IBM MQ* infrastructure.
3. **Unified Logging and Monitoring:** Centralizes monitoring via *AWS CloudWatch*, *AWS CloudTrail*, and *AWS OpenSearch Service* to capture end-to-end API audit trails.
4. **Security and Configuration Audit:** *AWS Config*, *AWS Security Hub*, and *AWS GuardDuty* perform automated compliance checks, threat detection, and key management across the entire ecosystem.

Comprehensive Solution Summary

The proposed architecture introduces a production-ready, cloud-native blueprint that successfully shifts traditional, monolithic banking operations into an agile, highly compliant Open Banking ecosystem. By decoupling public-facing API consumption from internal, mission-critical systems, the solution provides an optimized balance between security, scalability, and integration efficiency.

1. Multi-Tier Defense and Secure Zero-Trust Ingress

The perimeter security layout completely neutralizes edge-level threats before they touch the internal banking domain. The solution mandates mutual *TLS* (mTLS) for absolute token verification between external consumers, accredited third parties, and trust providers. Utilizing *AWS Shield* and *AWS WAF* at the entry points prevents large-scale *DDoS* attacks and application-layer injection exploits. Furthermore, separating edge routing

into a dedicated *Networking Account* isolates internal network mechanics, confining external *API connections* strictly to managed, secure *AWS API Gateway* instances.

2. Segmented Multi-Account Topology

To eliminate blast radiuses and ensure total operational governance, the platform implements a rigid multi-account strategy tied together via *AWS Transit Gateway* and *AWS PrivateLink*.

- **Core API Account:** Restricts microservice compute environments within highly isolated private networks.
- **Identity Provider Account:** Offloads authentication workloads to a isolated cluster supporting enterprise federation standards (*ForgeRock*, *Ping Identity*, *Keycloak*).
- **Developer Sandbox Account:** Isolates mockup environments and mock APIs so external fintech developers can test logic without accessing production data networks.

3. Decoupled Business Domains and Hybrid Data Synchronization

The application layer splits foundational open banking features into modular, microservices-driven pillars: *Account Information Services (AIS)*, *Payments Services (PIS)*, and automated webhooks. Instead of making risky, high-latency direct queries into physical banking cores for everyday requests, cloud transactions hit high-performance *AWS RDS* instances acting as an active cloud registry. Behind the scenes, secure *AWS Direct Connect* and *AWS VPN* channels continuously synchronize these cloud databases with physical mainframes, *Apache Kafka* clusters, and legacy message queues (*MQ*) located in the bank's traditional data center.

4. Automated Operational Auditing and Compliance Governance

To satisfy strict global financial regulations and zero-trust guidelines, the infrastructure leverages an array of automated monitoring tools. Centralized accounts run *AWS CloudWatch*, *AWS CloudTrail*, and *AWS X-Ray* to map real-time distributed tracing and compile immutable audit logs of every API call. Concurrently, security posturing tools like *AWS Config*, *AWS Security Hub*, and *AWS GuardDuty* perform continuous configuration scanning and active threat hunting, guaranteeing the platform remains consistently audit-ready without manual intervention.

5. Architectural Design Principles & Zero-Trust Framework

To meet the stringent data protection requirements of modern finance, the platform is built from the ground up on a *Zero-Trust Network Architecture* (ZTNA), enforcing continuous validation across every logical layer.

- **Principle of Least Privilege and Strong Isolation:** No component is granted implicit trust based on its network location. The multi-account organizational structure creates absolute cryptographic and network boundaries between the outer ingress zones, identity handling, sandboxed environments, and production database clusters.
- **End-to-End Cryptographic Enforcements:** All data in transit is encrypted using *TLS 1.3* externally and forced through secure *VPC Endpoints* internally. Data at rest within *AWS RDS*, *AWS S3*, and *AWS Secrets Manager* is encrypted using unique, customer-managed keys handled by *AWS Key Management Service (KMS)*, featuring automated rotation cycles.
- **Perimeter Throttling and Blast Radius Reduction:** By leveraging *AWS API Gateway* alongside *AWS WAF*, the platform implements granular rate limiting and circuit-breaker patterns. If a specific third-party provider experiences a breach or an accidental traffic spike, the system throttles that isolated endpoint automatically, preserving the availability of core banking domains.

6. Compliance Mapping & Regulatory Assurance

This framework functions as a direct technological match for global open banking mandates (such as Europe's *PSD2/PSD3*, the UK *Open Banking Standard*, and North America's *FDX guidelines*), shifting compliance from manual validation to automated enforcement.

- **Financial-Grade API (FAPI) Alignment:** The intersection of *mTLS* authentication at the edge gateways combined with dedicated *OAuth2/OIDC* Token Management (via *ForgeRock*, *Ping Identity*, or *Keycloak*) satisfies the strict *FAPI* security profiles mandatory for initiating high-value payment rails safely.
- **Granular Consent Tracking and Ledger Lifecycle:** The dedicated "Consents" datastore explicitly logs and tracks consumer permission lifecycles (granting, checking, and revoking access). This immutable transaction history directly fulfills global data privacy standards, including *GDPR*, *CCPA*, and localized banking confidentiality acts.

- **Continuous Configuration Auditing:** *AWS Config* continuously tracks system changes against pre-defined compliance rules, while *AWS Security Hub* consolidates security findings from *AWS GuardDuty* and *AWS Macie*. This ensures any drift from the certified, secure baseline is flagged, isolated, and remediated automatically before an auditor's evaluation.

End-to-End Customer Journey

This customer journey follows a single payment transaction from a user client or external payment authority through the modernized interface architecture. It demonstrates how individual AWS services activate chronologically to authorize users, execute isolated container services, stream real-time data logs, and synchronize records with legacy banking systems.

Phase 1: Request Ingress & Perimeter Defense

Step 1: The accredited *Third-Party Provider* (TPP) app initiates an API call on behalf of an authenticated user. *AWS Route 53* resolves the public *DNS*, and *AWS CloudFront* routes the request to the nearest edge location.

Step 2: At the edge boundary, *AWS Shield* instantly screens for volumetric network-layer *DDoS* attacks, while *AWS Web Application Firewall* (WAF) scans the request payloads to block application-layer injection vectors.

Step 3: The request reaches the *Network Account's* public boundary, where a handshake enforces Mutual *TLS* (mTLS) validation. This ensures the TPP possesses a valid cryptographic certificate issued by an approved trust authority.

Phase 2: Gateway Routing & Enterprise Identity Federation

Step 4: The mTLS-verified request lands on an *AWS API Gateway* instance. The *gateway* acts as a traffic cop, applying pre-configured throttling rules to prevent API abuse.

Step 5: The *API Gateway* triggers a custom authorizer that passes the user's bearer token through an *AWS PrivateLink* endpoint into the isolated Identity Provider Account.

Step 6: Inside the identity layer, an enterprise engine (such as *ForgeRock*, *Ping Identity*, or *Keycloak*) verifies the token's validity, decodes the associated *Financial-Grade API* (FAPI) claims, and checks that the user has granted active Consent to this TPP.

Phase 3: Core API Processing & Network Isolation

Step 7: Once identity and consent are validated, the *API Gateway* passes the request through *AWS Transit Gateway*, which securely bridges the isolation gap between the *Networking Account* and the internal *Open Banking API Account*.

Step 8: The traffic hits an internal *Load Balancer* and is assigned to the containerized *Account Information Service (AIS)* microservice running on highly available compute structures (like *AWS EKS* or *AWS ECS*).

Step 9: The *AIS* service checks local, high-speed metadata pools to confirm the exact data boundaries allowed by the active user consent profile.

Phase 4: Hybrid Data Retrieval & Synchronization

Step 10: To serve the balance query, the microservice pulls data from a low-latency, *cloud-native AWS RDS* instance operating as the primary, relational cloud registry.

Step 11: If the requested transactional data needs live re-verification from the system of record, a secure query is pushed over *AWS Direct Connect* or an *AWS Site-to-Site VPN* tunnel directly back to the physical banking infrastructure.

Step 12: Within the bank's on-premises data center, the query interfaces with message brokers like *Apache Kafka* or *IBM MQ* to communicate with legacy *mainframes*, retrieve the data, and return it safely to the cloud database layer.

Phase 5: Audit Logging & Clean Response Return

Step 13: Throughout the entire lifecycle, background processes like *AWS CloudTrail* and *AWS CloudWatch* log every infrastructure change and API invocation into an immutable, centralized bucket for compliance auditing.

Step 14: The microservice builds a compliant *JSON* response payload, hands it back through the secure transit routes, and *AWS API Gateway* delivers the finalized, encrypted financial data to the third-party provider's application.

Phase B: The TPP Onboarding Journey (Pre-Transaction)

This journey outlines how a *Third-Party Provider* establishes a trusted cryptographic handshake with your *AWS* environment before any customer-facing interactions can take place.

Step 1: The *external TPP* logs into the bank's developer portal (hosted in the *Developer Sandbox Account*) to test their software.

Step 2: Once certified, the *TPP* issues a *Dynamic Client Registration* (DCR) request over the public internet, passing an official *Software Statement Assertion* (SSA) signed by a *Central Open Banking trust directory*.

Step 3: The request enters the *Networking Account* via *AWS API Gateway* and is passed securely through *AWS PrivateLink* to the isolated *Identity Provider Account*.

Step 4: The *IAM* cluster (*ForgeRock* / *Ping Identity* / *Keycloak*) cryptographically validates the SSA, provisions a unique Client ID, attaches access parameters, and returns an encrypted registration token back to the TPP.

Phase C: The TPP Onboarding Journey (Pre-Transaction)

This asynchronous journey ensures that whenever an account balance changes or a payment clears, both the cloud cache and the third-party app are updated immediately without hammering the on-premises mainframe.

Step 1: A customer buys groceries using a physical debit card, triggering a balance change on the bank's physical *On-Premises Mainframe*.

Step 2: An on-premises data agent streams this delta event over *AWS Direct Connect*, updating the cloud-native *AWS RDS* repository instantly.

Step 3: The database updates trigger a data-event notification inside an event stream (like *AWS MSK* / *SQS*), which is immediately consumed by a containerized Webhook Worker service running on *AWS EKS*.

Step 4: The worker fetches the *TPP's* callback endpoint from the database and pushes an encrypted, asynchronous outbound event via the *NAT Gateway* in the *Networking Account*, instantly updating the user's third-party financial dashboard.

Phase D: The Consent Revocation Journey (Post-Transaction Lifecycle)

This journey fulfills data privacy laws (like *GDPR/FDX*), allowing a customer to completely sever ties with a *third-party* app and instantly wipe their security credentials.

Step 1: A customer opens their native mobile banking app and clicks "Revoke Access" next to a fintech app they no longer wish to use.

Step 2: The request hits the *Open Banking API Account*, where the application logic instantly updates the status of that permission block to REVOKED inside the Consent Datastore (*AWS RDS*).

Step 3: The microservice dispatches a high-priority token teardown signal to the *Identity Provider Account*.

Step 4: The *IAM* cluster immediately invalidates all active *OAuth2* Access and *Refresh Tokens* linked to that *TPP*-User pair and pushes the revoked token signatures into a distributed blacklist cache at the edge. Any subsequent API requests from that *TPP* are instantly blocked by *AWS API Gateway* with a 401 Unauthorized error.

Transaction Data-Flows

This section tracks the complete lifecycle of a customer transaction—specifically, an external *Third-Party Provider* (TPP) initiating an *Account Information Service* (AIS) request to view a customer's real-time balance. The walkthrough highlights exactly how traffic flows through the multi-tier *AWS* architecture and which services handle the request at each phase

Phase 1: Request Ingress & Perimeter Defense

Step 1: The accredited *Third-Party Provider* (TPP) app initiates an API call on behalf of an authenticated user. *AWS Route 53* resolves the public *DNS*, and *AWS CloudFront* routes the request to the nearest edge location.

Step 2: At the edge boundary, *AWS Shield* instantly screens for volumetric network-layer *DDoS* attacks, while *AWS Web Application Firewall* (WAF) scans the request payloads to block application-layer injection vectors.

Step 3: The request reaches the Network Account's public boundary, where a handshake enforces Mutual *TLS* (mTLS) validation. This ensures the TPP possesses a valid cryptographic certificate issued by an approved trust authority.

Phase 2: Gateway Routing & Enterprise Identity Federation

Step 4: The mTLS-verified request lands on an *AWS API Gateway* instance. The gateway acts as a traffic cop, applying pre-configured throttling rules to prevent API abuse.

Step 5: The *API Gateway* triggers a custom authorizer that passes the user's bearer token through an *AWS PrivateLink* endpoint into the isolated *Identity Provider Account*.

Step 6: Inside the identity layer, an enterprise engine (such as *ForgeRock*, *Ping Identity*, or *Keycloak*) verifies the token's validity, decodes the associated

Financial-Grade API (FAPI) claims, and checks that the user has granted active Consent to this TPP.

Phase 3: Core API Processing & Network Isolation

Step 7: Once identity and consent are validated, the *API Gateway* passes the request through *AWS Transit Gateway*, which securely bridges the isolation gap between the *Networking Account* and the internal *Open Banking API Account*.

Step 8: The traffic hits an internal *Load Balancer* and is assigned to the containerized *Account Information Service (AIS)* microservice running on highly available compute structures (like *AWS EKS* or *AWS ECS*).

Step 9: The *AIS* service checks local, high-speed metadata pools to confirm the exact data boundaries allowed by the active user consent profile.

Phase 4: Hybrid Data Retrieval & Synchronization

Step 10: To serve the balance query, the microservice pulls data from a low-latency, *cloud-native AWS RDS* instance operating as the primary, relational cloud registry.

Step 11: If the requested transactional data needs live re-verification from the system of record, a secure query is pushed over *AWS Direct Connect* or an *AWS Site-to-Site VPN tunnel* directly back to the physical banking infrastructure.

Step 12: Within the bank's on-premises *data center*, the query interfaces with message brokers like *Apache Kafka* or *IBM MQ* to communicate with legacy *mainframes*, retrieve the data, and return it safely to the cloud database layer.

Phase 5: Audit Logging & Clean Response Return

Step 13: Throughout the entire lifecycle, background processes like *AWS CloudTrail* and *AWS CloudWatch* log every infrastructure change and API invocation into an immutable, centralized bucket for compliance auditing.

Step 14: The microservice builds a compliant *JSON* response payload, hands it back through the secure transit routes, and *AWS API Gateway* delivers the finalized, encrypted financial data to the third-party provider's application.

Phase 6: Developer Onboarding & Dynamic Client Registration (DCR) Flow

Step 15: An external fintech developer enters through the *Developer Sandbox* Account to test APIs against mock environments using data isolated from production networks.

Step 16: When moving to production, the *TPP* app sends a registration request containing a *Software Statement Assertion* (SSA) signed by a central Open Banking directory.

Step 17: The public-facing *AWS API Gateway* catches this request and routes it directly to the *Identity Provider Account* via an *AWS PrivateLink* endpoint.

Step 18: The *IAM/Identity Engine* (*ForgeRock*, *Ping Identity*, or *Keycloak*) cryptographically validates the *SSA signature*, creates a client profile in its registry, and securely hands back a unique Client ID and registration token to the TPP.

Security Architecture

The platform implements a multi-layered, zero-trust security architecture aligned with *Financial-Grade API* (FAPI) profiles. By establishing rigid boundaries across network, identity, data, and operational layers, the framework ensures strict compliance with open banking regulations without compromising system performance.

1. Zero-Trust Boundary & Network Isolation

The architecture completely eliminates implicit trust by segmenting network access using distinct *AWS accounts* and *Virtual Private Clouds* (VPCs) connected through controlled channels.

- **Multi-Account Compartmentalization:** Segregates business capabilities into isolated *AWS accounts*—including dedicated *Networking*, *Identity Provider*, *Open Banking API*, and *Developer Sandbox accounts*—to minimize blast radiuses.
- **Private Integration Links:** Forbids internal microservices from exposing *public IP* addresses, utilizing *AWS PrivateLink* and internal *Network Load Balancers* (NLBs) for all cross-account traffic.
- **Hub-and-Spoke Interconnect:** Routes all intra-platform communications through a centralized *AWS Transit Gateway*, which acts as a secure, monitored router between accounts.

- **Secure Hybrid Backhaul:** Encrypts back-end connections to the bank's on-premises data center using redundant *AWS Direct Connect* and *AWS Site-to-Site VPN* lines.

2. Perimeter Ingress & Application Shielding

The edge routing layers act as a fortified gatekeeper, verifying external client credentials and deflecting malicious traffic before it can reach internal backend services.

- **Mutual TLS Handshake (mTLS):** Enforces mandatory *mTLS* verification at the public entry point of the *AWS API Gateway* to validate the digital certificates of accredited third parties against trust registries.
- **Layer 7 Exploitation Defense:** Deploys the *AWS Web Application Firewall* (WAF) directly at the ingress tier to inspect payloads and block cross-site scripting, *SQL injections*, and custom API anomalies.
- **Volumetric DDoS Mitigation:** Integrates *AWS Shield* at the global routing layer (*AWS Route 53* and *AWS CloudFront*) to intercept and absorb large-scale network floods automatically.
- **Granular Rate Throttling:** Implements structural rate-limiting rules within the *API Gateway* to prevent credential-stuffing attacks, denial-of-wallet attempts, and unauthorized API scraping.

3. Financial-Grade Identity Federation & Consent Management

Authentication and user authorization are completely decoupled from business computation, satisfying global privacy regulations and ensuring explicit user control.

- **Centralized Token Authority:** Offloads all *OAuth2/OIDC* lifecycle operations to an isolated *Identity Provider* account running enterprise engines like *ForgeRock*, *Ping Identity*, or *Keycloak*.
- **Decoupled Consent Datastore:** Archives and updates explicit user permission footprints within a restricted *AWS RDS* segment, ensuring data access matches the granted timeline.
- **Cryptographic Claim Validation:** Forces the *API Gateway* to process bearer tokens using decoupled custom authorizers, matching *FAPI* requirements before passing requests downstream.
- **Instant Token Lifecycle Revocation:** Syncs the consent store with an edge blacklist cache, immediately disabling access and refresh tokens the moment a user cancels permissions.

4. Continuous Auditing & Threat Intelligence

Operational compliance is enforced through automated monitoring systems that construct an immutable paper trail of configuration shifts and transaction flows.

- **Immutable Trail Ingestion:** Aggregates complete control plane audit events across all accounts via *AWS CloudTrail*, *funneling* logs into structured storage for compliance reviews.
- **Deep Observability Tracing:** Captures real-time transaction performance, microservice failures, and anomalous latency anomalies utilizing *AWS CloudWatch* and *AWS X-Ray*.
- **Automated Baseline Verification:** Runs *AWS Config* continuously to verify that resource deployments strictly match the corporate security baseline and financial industry regulations.
- **Intelligent Malice Detection:** Leverages *AWS GuardDuty* and *AWS Security Hub* to ingest continuous network metrics, flagging account anomalies and unauthorized network access patterns using machine learning.

Disaster Recovery (DR) Plan

To safeguard critical financial operations, this Open Banking architecture implements a robust *Multi-Region Active-Passive* Disaster Recovery (DR) strategy (aiming for a *Recovery Point Objective* (RPO) of near-zero for financial transactions and a *Recovery Time Objective* (RTO) of under 15 minutes). The plan decouples public API endpoints, compute workloads, stateful data, and hybrid connectivity to ensure continuous business operations during a primary region failure.

- **Recovery Point Objective (RPO):** near-zero for financial transactions (Maximum acceptable data loss)
- **Recovery Time Objective (RTO):** < 15 Minutes (Maximum acceptable downtime)

1. Traffic Routing & Edge Failover

The edge routing tier acts as the automated traffic cop, detecting regional health dips and instantly steering third-party providers (*TPPs*) to the backup infrastructure.

1. **Automated Route 53 Health Checks:** Deploys continuous synthetic monitors at the primary regional ingress boundary to track the health of public *AWS API Gateways*.

2. **Active-Passive DNS Failover:** Utilizes *AWS Route 53* failover routing policies to instantly flip the primary API domain's alias records to the secondary disaster recovery region upon health check failure.
3. **Global Edge Shielding Continuity:** Maintains a duplicated, hot-standby configuration of *AWS WAF* rules and *AWS Shield* properties in the secondary region to protect the backup endpoints from day one.
4. **mTLS Certificate Portability:** Replicates public trust registries and mutual *TLS* (mTLS) certificate validation schemas across both regions, ensuring TPPs do not experience handshake rejections during a failover event.

2. Multi-Region Data Replication & Consistency

Stateful customer balances and explicit consent permissions are continuously synchronized between regions using high-performance, asynchronous data pipelines.

1. **AWS Aurora Global Databases:** Operates an *AWS Aurora RDS* cluster with a primary write instance in the active region and a high-speed, cross-region read replica in the passive region, keeping cross-region replication latency under 1 second.
2. **Database Failover Promotion:** In a disaster scenario, an automated script or a single console command promotes the secondary read replica to a full read-write cluster, transforming it into the new primary system of record.
3. **Cross-Region Secret Management:** Syncs deployment configurations, cryptographic keys, and third-party validation secrets across regions using *AWS Secrets Manager* built-in multi-region replication capabilities.
4. **Immutable Backup Offloading:** Continuously copies automated database snapshots and transaction logs to a secure, write-once-read-many (WORM) *AWS S3* bucket in the secondary region, protecting data from accidental deletion or ransomware.

3. Compute Workload & Infrastructure Reconstitution

The platform leverages *Infrastructure as Code* (IaC) and containerization to instantly scale compute resources inside the secondary region when a disaster is declared.

1. **Pilot Light Compute Footprint:** Maintains a minimal, warm cluster configuration of *AWS EKS* (Elastic Kubernetes Service) or *AWS ECS* in the disaster recovery region to minimize standby costs.

2. **Automated Elastic Autoscaling:** Upon *DNS failover*, *AWS Auto Scaling* groups immediately scale the container fleet to match production workloads, pulling pre-built application images directly from a cross-region replicated *AWS ECR* registry.
3. **Automated IaC Reconstitution:** Standardizes the entire environment using *AWS CloudFormation* or *Terraform* templates, ensuring that the entire *multi-account* network topology can be torn down and rebuilt predictably within minutes.
4. **Decoupled Vault Isolation:** Ensures the *Identity Provider Account* (running *ForgeRock*, *Ping Identity*, or *Keycloak*) maintains its own active-passive database replication so user session tokens and access claims remain valid.

4. Hybrid Connectivity & Legacy Failover

Because the open banking cloud layer requires a continuous feed from physical banking cores, network pipelines back to the on-premises datacenter are engineered with total redundancy.

1. **Dual AWS Direct Connect Locations:** Establishes independent physical circuits into separate *AWS Direct Connect* locations, routing Link A to the primary region and Link B to the secondary region.
2. **Dynamic BGP Routing Failover:** Configures *Border Gateway Protocol* (BGP) weights so that internal on-premises message streams (*Kafka*, *IBM MQ*) automatically prefer the primary region path, swinging to the secondary path if Link A goes dark.
3. **Redundant Site-to-Site VPN Overrides:** Deploys an *AWS Site-to-Site VPN* backup tunnel over the public internet to serve as a low-cost, encrypted failover path if the physical Direct Connect infrastructure suffers an unexpected outage.
4. **Asynchronous Queue Buffering:** If connectivity to the on-premises core drops entirely, cloud microservices cache outbound notifications inside local *AWS SQS* queues, safely draining and processing the backlogged items once hybrid links are restored.

Architecture Tradeoffs

1. Security vs. Performance (API Latency)

- **The Trade-off:** Prioritizing absolute *Zero-Trust* boundary verification over raw API throughput and minimal network latency.
- **Evidence:** The architecture forces every single inbound transaction through a highly scrutinized, multi-tier gauntlet. Traffic must clear global routing (*AWS Route 53* and *AWS CloudFront*), pass two layers of inspection (*AWS Shield* and *AWS WAF*), execute a cryptographic handshake at the edge gateway (*mTLS*), cross account boundaries via *AWS Transit Gateway*, and perform out-of-band authorization checks against an isolated Identity Provider Account via *AWS PrivateLink* before it ever touches a core business microservice. This defense-in-depth layout intentionally adds 15–40 milliseconds of processing overhead per request compared to a monolithic structure, sacrificing peak performance to ensure absolute data isolation and protection.

2. Microservice Autonomy vs. Legacy Operational Complexity

- **The Trade-off:** Decoupling the cloud application layer into nimble microservices while increasing the complexity of stateful synchronization back to the legacy core.
- **Evidence:** The architecture establishes three distinct cloud-native microservice pillars inside the *Open Banking API* Account (Account Information Services, Payment Services, and Webhooks) backed by cloud registries (*AWS RDS*). However, because financial records must ultimately reconcile with the physical data center, this introduces a permanent data synchronization burden. The system relies on heavy middleware orchestration—linking cloud storage back to physical *Apache Kafka* clusters, *IBM MQ* infrastructure, and mainframe environments over hybrid connectivity pipelines (*AWS Direct Connect*). This grants the cloud team tremendous development agility, but requires complex distributed system management to handle eventual consistency, network partition risks, and dual-system reconciliation.

3. Strict Compliance Guardrails vs. Operational Cost Inflation

- **The Trade-off:** Implementing an aggressive multi-account governance framework that maximizes compliance assurance but increases base cloud infrastructure spending.
- **Evidence:** Instead of grouping components within a single, cost-efficient *AWS account* using logical *network partitions*, this architecture mandates a strict physical separation of concerns. It splits operations across a dedicated *Networking Account*, an *Identity Provider Account*, an *Open Banking API Account*, and a *Developer Sandbox Account*. Each of these isolated zones requires duplicate baseline resources, including its own *Virtual Private Clouds (VPCs)*, highly available *NAT Gateways* spread across multiple *Availability Zones*, standalone *AWS API Gateways*, and dedicated automated governance tools (*AWS Config*, *AWS CloudTrail*, and *AWS Security Hub*). This *multi-account* posture dramatically simplifies financial regulatory audits but creates a high baseline operational cost floor, regardless of actual transactional traffic volume.

4. Disaster Recovery Resilience vs. Cross-Region Sync Overhead

- **The Trade-off:** Optimizing for a near-zero transaction Recovery Point Objective (RPO) by introducing cross-region database lock risks and replication costs.
- **Evidence:** To satisfy the high-availability disaster recovery blueprint, the system deploys an *AWS Aurora Global Database* architecture that mirrors data continuously between an active primary region and a passive secondary target. While this guarantees that user consents and balances are preserved during a total regional outage, it relies on asynchronous *cross-region storage replication*. If a primary cluster goes down mid-transaction, engineers face a data consistency dilemma: either wait for the cross-region replication lag to fully catch up (inflating the Recovery Time Objective) or forcefully promote the replica and risk data loss. Additionally, maintaining multi-region secrets, replicated *KMS* keys, and warm container standbys (*AWS EKS/ECS* pilot light footprints) requires paying for idle, premium infrastructure around the clock.

5. Local Data Sovereignty vs. Standardized Cloud Scalability

- **The Trade-off:** Restricting data residency options to strictly comply with sovereign regional banking laws rather than auto-routing workloads to the most cost-efficient global cloud infrastructure.
- **Evidence:** The system configuration binds all data ingestion pipelines and processing nodes exclusively within a single geographic territory's local *Availability Zones (AZs)* to satisfy regulations like India's RBI storage directives, Europe's GDPR, or Saudi Arabia's SAMA framework. The architecture actively rejects the standard cloud strategy of dynamic, borderless multi-region workload balancing. Instead of utilizing cheap, cross-border spot instances or overflow capacity from neighboring AWS *regions* during peak traffic spikes, the platform is forced to scale vertically and horizontally strictly within its localized cluster bounds. This guarantees airtight compliance and eliminates data-export litigation risks, but significantly increases the institutional baseline price-per-compute unit and limits emergency failover alternatives to identical local availability regions.

Architecture Decision Records

1 Multi-Account AWS Landing Zone Strategy

- **Decision:** Separate the platform infrastructure across four distinct, dedicated *AWS Accounts: Networking Ingress, Identity Provider, Open Banking API, and Developer Sandbox*.
- **Rationale:** This design pattern enforces absolute isolation between untrusted public code testing, user credential validation, and production transaction processing. It limits the blast radius of any individual account compromise, simplifies financial regulatory audits, and allows security teams to apply strict, decoupled *Identity and Access Management (IAM)* permissions per business function.

2. Mandatory Mutual TLS (mTLS) at the Edge Ingress Tier

- **Decision:** Enforce a cryptographic *mTLS* handshake directly at the public-facing *AWS API Gateway* for all third-party provider (*TPP*) communication.
- **Rationale:** *Standard API* token verification is insufficient for *Financial-Grade API (FAPI)* profiles. Handshaking via *mTLS* checks the third party's digital certificate directly against open banking trust directories at the perimeter, ensuring absolute identity validation and blocking

unauthorized connection attempts before requests hit internal microservices.

3. Centralized Hybrid Connectivity Hub via AWS Transit Gateway

- **Decision:** Route all cross-account traffic and hybrid on-premises backhaul (*AWS Direct Connect/VPN*) through a centrally managed *AWS Transit Gateway*.
- **Rationale:** Managing individual point-to-point *Virtual Private Cloud (VPC)* peering connections quickly becomes unmanageable and highly prone to security configuration errors. A centralized *hub-and-spoke* topology unifies *firewall policy enforcement*, simplifies route tables, and creates a single, audit-ready checkpoint for all internal corporate traffic.

4. Decoupled Identity and Consent Management Architecture

- **Decision:** Offload user authentication and consent state validation to an isolated identity tier running enterprise software (*ForgeRock, Ping, or Keycloak*) separate from business application logic.
- **Rationale:** Separating identity checks from core payment logic protects customer credentials from application-layer bugs. This layout prevents core banking APIs from ever seeing raw login details, allows identity microservices to scale independently during traffic spikes, and provides a specialized datastore to log explicit consumer permissions for compliance.

5. AWS Aurora PostgreSQL with Cross-Region Global Databases

- **Decision:** Utilize *AWS Aurora Global Databases* as the primary relational cloud registry, maintaining an active writer cluster in the main region and an asynchronous read replica in a secondary region.
- **Rationale:** *Open Banking* systems require low-latency read speeds alongside strict transaction accuracy (*ACID compliance*). *Aurora* delivers high throughput for account balance checks, while its global database capabilities keep cross-region replication lag under one second, meeting tough disaster recovery benchmarks (*RPO < 1 second*).

6. Multi-Layer Border Security with AWS Shield and AWS WAF

- **Decision:** Deploy *AWS Shield Advanced* at the *DNS routing* layer and apply specialized *AWS Web Application Firewall (WAF)* rule groups onto the public *AWS API Gateway*.

- **Rationale:** Exposing financial endpoints to the public internet attracts frequent cyberattacks. This layered combination ensures that large network-layer *DDoS* attacks are deflected automatically by *Shield*, while the *WAF* filters out application-layer hazards like *SQL injections*, *Cross-Site scripting*, and unauthorized *API scraping*.

7. Read-Optimized Cloud Caching Engine

- **Decision:** Implement an intermediate data replication loop using *Change Data Capture (CDC)* to build a synchronized clone of core on-premises data inside the cloud database layer.
- **Rationale:** Direct, real-time queries to legacy mainframes are slow, expensive, and risk overloading the bank's core systems during peak hours. Serving customer requests from an optimized cloud clone dramatically cuts API response times and keeps the core infrastructure safe from unexpected public traffic surges.

8. Containerized Microservices via AWS EKS/ECS

- **Decision:** Deploy all open banking functional capabilities (Account Information Services, Payment Services, and Webhooks) as containerized workloads managed by a container orchestration platform.
- **Rationale:** Financial regulations and fintech partner needs change rapidly. Using containers allows development teams to build, test, and update individual banking services independently without taking down the entire platform, while autoscaling groups ensure the system scales smoothly during morning rush hours.

9. Centralized, WORM-Compliant Logging Architecture

- **Decision:** Route all infrastructure audits, microservice execution logs, and API tracking events into an immutable, centralized *AWS S3* bucket running *Write-Once-Read-Many (WORM)* object locking.
- **Rationale:** Financial regulators require untampered, permanent transaction audit trails. Centralizing logs using *AWS CloudTrail* and *AWS CloudWatch*—and locking them against deletion—ensures compliance with data regulations, simplifies forensic reviews, and prevents compromised accounts from erasing historical tracks.

10. Private Cross-Account Integration using AWS PrivateLink

- **Decision:** Connect the public *AWS API Gateway* to internal microservices and isolated identity clusters exclusively via *AWS PrivateLink* and internal *Network Load Balancers* (NLBs).
- **Rationale:** Traditional routing using public IP addresses or complex *VPC peering* exposes internal communication to security risks. *PrivateLink* ensures that sensitive authentication data and transaction payloads move entirely within the private *AWS network backbone*, never exposing core financial services to the public internet.

AWS Well-Architected Framework Review

An architectural assessment of this Open Banking platform against the *AWS Well-Architected Framework* reveals a highly structured, enterprise-grade design pattern.

1. Security

Strengths

- **Airtight Zero-Trust Segmentation:** The deployment of a dedicated, multi-account *Landing Zone* (*Networking*, *Identity*, *API*, and *Sandbox* accounts) ensures strict separation of duties and slashes the attack blast radius.
- **Financial-Grade Edge Isolation:** Forcing all inbound traffic through a perimeter of *AWS Shield Advanced*, *AWS WAF*, and mandatory *mutual TLS* (*mTLS*) handshake verification perfectly aligns with *Financial-Grade API* (FAPI) security profiles.
- **Private Transit Paths:** Utilizing *AWS PrivateLink*, internal *Network Load Balancers* (NLBs), and *AWS Transit Gateway* ensures that sensitive financial transactions and token exchanges bypass the public internet entirely.

Recommendations

- **Automate Secret and Key Rotation:** Implement automated, zero-downtime rotation inside *AWS Secrets Manager* and *AWS KMS* for all cryptographic keys and identity provider backend certificates to minimize credential-exposure windows.
- **Incorporate Data Loss Prevention (DLP):** Deploy *AWS Macie* over the logging and *AWS RDS* storage layers to automatically scan for and alert on accidental leaks of *Personally Identifiable Information* (PII) or unencrypted account balances.

2. Reliability

Strengths:

- **Highly Available Cloud Data Layer:** Implementing *AWS Aurora* Global Databases with cross-region read replicas ensures low-latency operations and a near-instant switchover target if a region fails.
- **Decoupled Architecture Pillars:** Breaking down application services into containerized microservices (*AIS, PIS, Webhooks*) on *AWS EKS/ECS* prevents a failure in one application tier from crashing other domains.
- **Resilient Multi-AZ Networking:** *Multi-AZ* deployments of *NAT Gateways* and *Application Load Balancers* provide automatic, localized infrastructure failover.

Recommendation:

- **Incorporate Circuit-Breaker Patterns:** Introduce a service mesh (like *AWS App Mesh* or *Istio*) to implement circuit-breakers and back-off throttles on *API endpoints*, preventing systemic failures when the on-premises legacy data center faces high latency or drops offline.
- **Conduct Automated Chaos Simulations:** Run regular, automated disruption testing using *AWS Fault Injection Service (FIS)* to simulate cross-region database failovers, testing whether your RTO (< 15 mins) holds up under stress.

3. Performance Efficiency

Strengths:

- **Read-Optimized Asynchronous Synchronization:** Building a *cloud-native* clone of core system records in *AWS Aurora RDS* via a background *Change Data Capture (CDC)* loop prevents slow *legacy mainframes* from bottlenecking public API response speeds.
- **Edge-Optimized Content Routing:** Utilizing *AWS Route 53* and *AWS CloudFront* guarantees third-party requests are processed at the closest global edge location.

Recommendations

- **Introduce an In-Memory Caching Tier:** Deploy an *AWS ElastiCache* (Redis OSS) cluster directly in front of the *Open Banking API* layer to intercept frequent account balance checks, reducing database CPU load and slicing API latency down to single-digit milliseconds.
- **Utilize Compute Autoscaling Metrics:** Configure custom *AWS CloudWatch* metric alarms (tracking container memory pressure or API request counts,

rather than standard CPU metrics) to spin up *AWS EKS* pods proactively before high-volume banking traffic spikes hit.

4. Cost Optimization

Strengths:

- **Isolated Testing Sandbox:** Establishing a standalone *Developer Sandbox account* prevents external fintech testing traffic from consuming production database *IOPS* or driving up live infrastructure expenses.
- **Managed Gateway Services:** Relying on serverless and managed components (*AWS API Gateway, Aurora RDS*) removes the overhead costs of manually scaling underneath baseline server operating systems.

Recommendation:

- **Implement VPC Endpoint Optimization:** Consolidate redundant *cross-AZ VPC Endpoints* and audit *Transit Gateway* data processing fees, as unoptimized cross-account data streaming can inflate your operational baseline costs.
- **Leverage Compute Savings Plans:** Commit to an *AWS Compute Savings Plan* for your steady-state *AWS EKS* compute nodes, and configure *Aurora Auto Scaling* to dynamically spin down non-critical development resources outside of standard business working hours.

5. Operational Excellence

Strengths:

- **Immutable Compliance Audit Logs:** Routing infrastructure and application traces through *AWS CloudTrail, AWS CloudWatch*, and *AWS Config* directly into a *WORM-locked AWS S3* bucket guarantees clean, tamper-proof regulatory paper trails.
- **Standardized Environments:** Utilizing *Infrastructure as Code (IaC)* ensures that *multi-account networks, VPC setups*, and *firewall* structures remain entirely identical between production and disaster environments.

Recommendation:

- **Transition to GitOps Workflows:** Establish automated *GitOps* deployment pipelines using *AWS CodePipeline* alongside tools like *ArgoCD* to guarantee that zero manual infrastructure modifications occur directly within production environments.
- **Build Unified Operational Dashboards:** Combine *AWS CloudTrail* event loops with application logs inside *AWS OpenSearch Service* to construct

single-screen dashboards tracking end-to-end transaction latency and third-party error distributions.

6. Sustainability

Strengths:

- **Efficient Compute Densities:** Deploying workloads via containerized runtimes (*EKS/ECS*) improves server utilization rates and dramatically slashes computing waste compared to running dedicated virtual machines.
- **Managed Platform Efficiencies:** Handing infrastructure optimization tasks off to AWS managed services (such as *Aurora PostgreSQL*) ensures the platform benefits from optimized hardware resource allocation.

Recommendation:

- **Migrate Compute Workloads to AWS Graviton:** Transition *AWS EKS* compute pools and *AWS Aurora* database clusters over to *AWS Graviton* processors to immediately reduce power consumption and improve carbon efficiency profiles by up to 60%.
- **Establish Lifecycle Data Expiry Policies:** Enact strict data lifecycle management rules inside *AWS S3* to archive old application logs and ancient audit histories onto cold *AWS Glacier Flexible Retrieval* storage tiers, minimizing active disk spin footprints.

Architecture Risks

1. Inconsistent Cloud-to-On-Premises Data States

- **Flaw:** Relying on asynchronous *Change Data Capture* (CDC) or event-driven streams to sync the on-premises database with the cloud *AWS RDS* instance.
- **Risk:** A customer could complete a balance inquiry or a second transaction while the sync pipeline is lagging, leading to data race conditions, double-spending vulnerabilities, and inaccurate financial reporting.
- **Fix:** Implement an intermediate read-through cache strategy using *AWS ElastiCache* combined with conditional real-time *API* callouts. If a high-value transaction is initiated, the microservice must force an inline validation check directly against the on-premises system of record over *AWS Direct Connect* before approving the request.

2. Cross-Region Storage Replication Lag During Disaster Failover

- **Flaw:** Relying on asynchronous block-level replication between the primary *active database region* and the passive disaster recovery region within *AWS Aurora Global Databases*.
- **Risk:** If the primary *AWS region* experiences an abrupt, catastrophic failure, any transaction data in flight during the sub-second replication window will be lost, violating strict zero-data-loss banking regulations ($RPO > 0$).
- **Fix:** Force all critical payment write transactions to log an immutable backup entry onto an active-active message ledger or a globally replicated *AWS DynamoDB* table with Global Tables enabled before returning a success code to the user.

3. Legacy Connectivity Exhaustion Under High Traffic Spikes

- **Flaw:** Exposing containerized cloud microservices (*AWS EKS/ECS*) that autoscale dynamically to an on-premises core datacenter backend that relies on fixed-capacity connection pools (*IBM MQ* or *mainframes*).
- **Risk:** During sudden traffic spikes, the rapidly scaling cloud layer will flood the physical data center with requests, overloading legacy firewalls, exhausting message queues, and knocking out core banking services for both cloud and physical branches.
- **Fix:** Deploy *AWS SQS* queues or an *Apache Kafka* buffering layer (*AWS MSK*) between the cloud microservices and the *AWS Transit Gateway*. This design pattern throttles outbound legacy requests, converting unpredictable spikes into a smooth, predictable stream that matches the on-premises processing capacity.

4. Shared Credentials and Blast Radius in Secrets Manager

- **Flaw:** Consolidating database credentials, third-party cryptographic keys, and *OAuth2* client secrets within a single, centrally accessible instance of *AWS Secrets Manager*.
- **Risk:** A single loose Identity and Access Management (IAM) policy or a compromise within an application container could allow an attacker to dump the entire vault, exposing both cloud databases and on-premises core *network keys*.
- **Fix:** Separate vaults across the *multi-account* structure using isolated, account-specific *AWS Secrets Manager* deployments. Enforce strict

resource-based *IAM policies* paired with unique, customer-managed keys in *AWS KMS* that require distinct decryption contexts per microservice.

5. Denial-of-Wallet Exploits via Untrusted Third-Party Endpoints

- **Flaw:** Allowing public-facing *AWS API Gateway* instances to accept un-throttled *Dynamic Client Registration* (DCR) or heavy search API calls from accredited *third-party providers* (TPPs).
- **Risk:** Malicious or poorly optimized third-party applications can launch distributed, high-volume API requests that balloon the bank's transactional *AWS utility billing* (*API Gateway*, *NAT Gateway*, data transfer fees) without triggering classic network-layer *DDoS* protections.
- **Fix:** Enforce granular consumption tiers and strict usage plans within the *AWS API Gateway*. Apply restrictive rate-limiting and throttling metrics mapped directly to the individual TPP's validated certificate claims at the *AWS WAF* boundary.

6. Single Point of Failure at the Central Identity Provider

- **Flaw:** Offloading all *Financial-Grade API* (FAPI) token checks and user consent lookups to a single cluster running in the isolated Identity Provider Account.
- **Risk:** If the *IAM* cluster (*ForgeRock*, *Ping*, or *Keycloak*) crashes or experiences severe network latency, the entire Open Banking ecosystem will freeze instantly, as the edge gateways will be unable to validate incoming tokens.
- **Fix:** Run the identity infrastructure across multiple *Availability Zones* using high-density autoscaling groups backed by a dedicated *multi-region* instance of *AWS Aurora*. Implement a read-only cache token validation loop directly at the edge layer using *AWS Lambda@Edge* to offload repetitive signature checking.

7. Stale Token Revocation Footprints at the Cloud Edge

- **Flaw:** Relying on standard token *Time-To-Live* (TTL) durations to naturally expire user access keys after a customer revokes app permissions inside their mobile banking portal.

- **Risk:** After clicking "Revoke Access," a third-party app could continue to pull sensitive balance sheets and account histories for minutes or hours until its active bearer token naturally expires, violating strict data privacy laws (*GDPR/FDX*).
- **Fix:** Establish an instantaneous backchannel event loop. The moment a user cancels consent, the application layer must publish a revocation signal to an in-memory database cluster like *AWS ElastiCache* that is queried by the edge *AWS API Gateway* custom authorizers before every single API call.

8. Certificate Revocation List (CRL) Outages at the mTLS Ingress

- **Flaw:** The edge *AWS API Gateway* checking the validity of incoming *TPP certificates* against external *Open Banking* trust directories or *Certificate Authority* (CA) servers in real time.
- **Risk:** If the external trust directory's revocation checking mechanism goes offline or experiences high network latency, the edge gateway will either drop all valid connection handshakes (fail-closed) or accidentally let revoked, malicious certificates through (fail-open).
- **Fix:** Cache the external Certificate Revocation Lists (CRLs) and Online Certificate Status Protocol (OCSP) responses locally inside an internal, high-availability data bucket updated every few minutes by an automated background worker thread via *AWS Lambda*.

9. Unencrypted PII Contamination in Log Aggregators

- **Flaw:** Standardizing end-to-end distributed system tracing across microservices using *AWS CloudWatch*, *AWS X-Ray*, and *AWS CloudTrail* without data inspection policies.
- **Risk:** Developer oversight or default error-handling dumps can write unencrypted customer account numbers, real-time balances, and legal names directly into plain-text system logs, exposing sensitive data to internal staff and violating compliance standards (*PCI-DSS/GDPR*).
- **Fix:** Embed automated data scrubbing libraries directly inside the application container code templates. Additionally, activate *AWS Macie* or configure *AWS CloudWatch Logs Data Protection* policies to scan, mask, and redact sensitive financial matching patterns before logs are written to disk.

10. Direct Connect Failover Disruption and Route Flapping

- **Flaw:** Configuring redundant *AWS Direct Connect* locations without matching *Border Gateway Protocol* (BGP) routing weights or robust failover prioritization metrics.
- **Risk:** During a minor network disruption or packet drop event, the routing paths can begin to rapidly swing back and forth ("route flapping") between the primary link and the backup connection, degrading network performance and corrupting *real-time streaming data pipelines*.
- **Fix:** Establish a strict active-passive hybrid path design. Configure explicit asymmetric BGP communities, setting higher local preference values on the primary *AWS Direct Connect* route while mapping an automated *AWS Site-to-Site VPN* backup path with a significantly lower routing priority to handle emergency overflow traffic smoothly.

Architectural Recommendations

1. Transition Compute Clusters to ARM64 Architecture

- **Action:** Migrate all containerized open banking microservices (*AIS*, *PIS*, *Webhooks*) from *x86-based* compute instances to *AWS Graviton* processors.
- **Implementation:** Update the underlying *AWS EKS* node groups to use *Graviton* instances and modify the *CI/CD* pipeline to compile container images using multi-architecture build setups. This transition will immediately cut compute infrastructure costs by up to 20% and reduce carbon output while improving price-per-performance metrics for encryption-heavy financial *Java* and *Node.js* microservices.

2. Introduce an Edge-Level Token Validation Cache

- **Action:** Move repetitive *OAuth2* Access Token validation out of the core application network and up to the global routing tier.
- **Implementation:** Deploy *AWS Lambda@Edge* or *AWS CloudFront* Functions at the *AWS CloudFront* distribution layer, paired with an edge-optimized *AWS ElastiCache* cluster. The edge function intercepts incoming requests, verifies token signatures, and cross-references active token IDs against a distributed blacklist, completely deflecting expired or illegitimate requests before they incur data costs at the *API Gateway*.

3. Establish a Resilient Circuit Breaker and Retry Fabric

- **Action:** Implement a localized fault-isolation and network stabilization layer to handle intermittent latency spikes from the on-premises *mainframe*.
- **Implementation:** Deploy a service mesh framework like *AWS App Mesh* or *Istio* across the *AWS EKS* compute cluster. Configure distinct virtual routers with circuit-breaker patterns and exponential back-off retry budgets on all outbound connections to the hybrid network, ensuring that downline core slowdowns do not choke internal cloud resources.

4. Automate Continuous Secret and Certificate Rotation

- **Action:** Eliminate static, long-lived operational credentials and trust certificates across all multi-account boundaries.
- **Implementation:** Configure *AWS Secrets Manager* to natively manage database credentials and external *API* keys, utilizing built-in *AWS Lambda* rotation functions to cycle them every 30 days. For internal *mTLS* connections, integrate *AWS Private CA* with automated *Kubernetes certificate managers* to automatically renew and push short-lived pod-to-pod identity certificates.

5. Enforce Real-Time PII Log Scrubbing and Data Protection

- **Action:** Prevent accidental writing of customer bank account numbers and raw financial balances into centralized plain-text logs.
- **Implementation:** Configure *AWS CloudWatch Logs Data Protection* policies across all production log groups. This native masking engine automatically scans incoming log streams using machine learning to detect, redact, and alert on any financial markers or *Personally identifiable information* (PII) before it is permanently stored.

6. Optimize Hybrid Connectivity with a Backup Cloud WAN Hub

- **Action:** Enhance network routing resiliency and eliminate single-point-of-failure risks on the core datacenter connection lines.
- **Implementation:** Interconnect the *AWS Transit Gateway* with *AWS Cloud WAN* to establish a global core network framework. Map redundant *AWS Direct Connect* links as the primary path, and deploy *AWS Site-to-Site VPN* connections running dynamic *Border Gateway Protocol* (BGP) routing as a

hot-standby path to automatically take over traffic if the physical circuit is severed.

7. Deploy Automated Machine-Learning Threat Ingestion

- **Action:** Shift from manual, reactive security log auditing to automated, proactive anomaly detection across the multi-account landing zone.
- **Implementation:** Activate *AWS GuardDuty* across all accounts and integrate it directly with *AWS Security Hub*. Configure *GuardDuty* to continuously ingest *VPC Flow Logs*, *DNS queries*, and *CloudTrail* management events, utilizing its embedded machine learning models to instantly isolate and block compromised containers attempting out-of-character lateral movements.

8. Implement an Event-Driven Outbound Webhook Buffer

- **Action:** Protect external fintech partners from missing asynchronous settlement updates during peak traffic or vendor platform outages.
- **Implementation:** Route all outgoing notification events generated by the containerized webhook worker into a highly scalable *AWS SQS Dead-Letter Queue (DLQ)* system. If a third-party application's server drops offline, the queue automatically retains the event and retries delivery using an exponential back-off strategy rather than discarding the message.

9. Enact WORM Storage Lifecycles for Regulatory Compliance

- **Action:** Fulfill rigid data sovereignty and audit-trail preservation laws by locking infrastructure audit logs against deletion or modification.
- **Implementation:** Enable *S3 Object Lock* in *Write-Once-Read-Many (WORM)* Compliance mode on the centralized bucket that holds data from *AWS CloudTrail* and *AWS Config*. Pair this with *S3 Lifecycle* policies that systematically migrate log records into *AWS S3 Glacier Flexible Retrieval* after 90 days to minimize ongoing active disk costs.

10. Establish GitOps-Driven Infrastructure as Code Compliance

- **Action:** Eradicate manual adjustments ("configuration drift") within the production environment to guarantee a completely predictable disaster recovery footprint.
- **Implementation:** Model the entire multi-account architecture layout using *AWS CloudFormation* or *Terraform* modules. Force all modifications

to clear a Git-based repository pipeline using tools like *AWS CodePipeline*, ensuring that manual infrastructure edits via the *AWS Management Console* are permanently blocked by *Service Control Policies* (SCPs).

Boundary Conditions for Reference Architecture

1. Ingress Rate Limiting and Volumetric Thresholds

- **The Issue:** Uncontrolled traffic spikes or distributed denial-of-service (DDoS) attempts from third-party applications can overwhelm edge compute resources and saturate downstream internal networks.
- **The Condition:** The perimeter *AWS API Gateway* tier must enforce a strict maximum ceiling of 10,000 requests per second (RPS) per accredited *Third-Party Provider* (TPP) identifier, with a hard burst capacity cap of 15,000 RPS. Any external traffic exceeding this configured threshold must be instantly deflected at the *AWS WAF* boundary with an immediate HTTP 429 Too Many Requests response code to protect internal systems from resource exhaustion.

2. Cross-Region Storage Replication Sync Latency Limits

- **The Issue:** High data-replication lag between geographic territories risks creating data splits and consistency issues if an emergency failover is triggered while the secondary cluster is behind.
- **The Condition:** The asynchronous block-level data streaming pipeline within the *AWS Aurora* Global Database engine must maintain a cross-region replication lag threshold of under 1,000 milliseconds. If the replication latency exceeds this 1-second limit due to cross-border fiber disruptions, the automated disaster recovery monitoring scripts must block any failover promotions to avoid data loss, defaulting to a read-only state in the secondary zone instead.

3. Legacy Datacenter Connection Pool Bottlenecks

- **The Issue:** Dynamically autoscaling cloud container fleets (*AWS EKS/ECS*) can easily flood the fixed-capacity connection queues of on-premises mainframes, knocking out traditional banking cores.
- **The Condition:** The outbound integration middleware layer traversing the *AWS Transit Gateway* must restrict concurrent connections to the physical on-premises network to a hard maximum of 2,500 active parallel sessions across the entire cloud fleet. If cloud traffic scales past this point, the extra

outbound database requests must be queued up inside an *AWS SQS* buffer layer rather than spawning new live connections back to the datacenter.

4. Financial-Grade Token Lifecycle and Expiry Enforcements

- **The Issue:** Stale or long-lived authentication keys violate global financial security guidelines and leave a wide window open for unauthorized data access.
- **The Condition:** The isolated *Identity Provider Account* must restrict the validity window of *OAuth2* Access Tokens to a hard ceiling of exactly 600 seconds (10 minutes), while *Refresh* Tokens must expire within 24 hours. Furthermore, if a customer revokes app access inside their mobile banking portal, the revocation signal must update the edge cache within 500 milliseconds to instantly block subsequent requests.

5. Data Sovereignty and Regional Storage Boundaries

- **The Issue:** Storing or backing up sensitive financial records outside local country borders violates national banking privacy regulations and exposes the institution to severe legal penalties.
- **The Condition:** The data lifecycle policies across all *AWS RDS* instances, *AWS S3* buckets, and *AWS CloudWatch* log groups must explicitly restrict data movement within a single designated geographic *AWS region* (e.g., *ap-south-1* or *eu-west-1*). The cross-account governance model must enforce strict *Service Control Policies* (SCPs) that block any attempts to replicate encrypted backups or archive log files to data buckets outside local sovereign borders.

6. Mutual TLS Certificate Validation Check Timeouts

- **The Issue:** Slow cryptographic certificate checking at the edge gateway drastically increases overall API latency, hurting the end-user experience.
- **The Condition:** The *mutual TLS (mTLS)* verification loop executed by the *AWS API Gateway* against external *Open Banking* directory Certificate Revocation Lists (CRLs) must complete in under 150 milliseconds. If an external trust registry directory goes offline or fails to respond within this 150ms timeout window, the edge gateway must fallback to a locally cached copy of the certificate directory to maintain fast response speeds.

7. On-Premises Event Streaming Buffer Capacity Constraints

- **The Issue:** An extended network outage on the physical connection links can cause a backlog of transaction logs that will eventually exhaust cloud storage pools.
- **The Condition:** The local cloud data ingestion layers (*AWS MSK / Apache Kafka* clusters) must have enough storage allocated to handle a complete *AWS Direct Connect* line outage for a maximum duration of 48 hours at peak volume. If the hybrid connection lines stay down longer than 48 hours, the cloud cache buffers will fill up, requiring the platform to stop accepting new payment requests to avoid data corruption.

8. Container Compute Cluster Autoscaling Limits

- **The Issue:** Uncapped application pod autoscaling can consume all available IP addresses in a *Virtual Private Cloud* (VPC), blocking other internal services from running.
- **The Condition:** The *AWS EKS* worker node scaling groups inside the *Open Banking API* Account must operate within a strict scale-out boundary of between 5 and 50 concurrent active virtual server instances across multiple availability zones. This ceiling prevents a runaway application bug from causing an ip-address exhaustion failure within the private subnet allocations.

9. Cryptographic Key Lifecycle and Multi-Region Portability

- **The Issue:** A failure to replicate *encryption keys* across regions before data arrives will leave the backup disaster recovery system completely unable to read the database.
- **The Condition:** Every single customer-managed cryptographic key handled inside *AWS KMS* must be explicitly deployed as a *Multi-Region Key*. This setup ensures that the exact same key material and key ID are mirrored across both regions simultaneously, allowing the secondary standby database to decrypt storage volumes instantly if the primary region goes offline.

10. Audit Log Compliance Retain-and-Lock Windows

- **The Issue:** Early deletion or tampering of historical infrastructure traces breaks strict financial auditing compliance rules.

- **The Condition:** The centralized *AWS S3* audit log bucket must enforce an unbreakable compliance retention window of exactly 2,555 days (7 years) using native *S3* Object Locking in *WORM* compliance mode. This hard condition prevents any user—including root administrators and compromised automated deployment pipelines—from deleting or modifying infrastructure traces until the legal retention period has passed.

Appendix A: Non-Functional Requirements (NFR) Matrix

This matrix defines the critical quality attributes, operational constraints, and performance benchmarks that the system must satisfy to ensure long-term stability, security, and scalability.

NFR Category	Target Objective	Engineering Validation Mechanism(Tools)	Architecture Component Mapping
Availability	99.99% Uptime for Core APIs. 99.999% Uptime for edge routing.	- Synthetic Monitoring AWSCloudWatch Synthetics. - Chaos testing via AWS Fault Injection Service (FIS).	Multi-AZ deployments of AWS API Gateway, Application Load Balancers, and container clusters.
Latency	- Edge processing < 50ms. - Backend processing < 150ms (excluding legacy sync dependencies).	- Real-time tracing via AWS X-Ray - Performance testing via Apache JMeter / Gatling workloads.	Edge routing (AWS CloudFront), AWS API Gateway, and containerized services (AWS EKS/ECS).
Data Protection	100% encryption for data at rest and in transit. - Zero plain-text PII leaks.	- Automated configuration checks via AWS Config. - Machine learning PII scanning via AWS Macie.	AWS KMS (Customer-Managed Keys), AWS Secrets Manager, and AWS RDS storage volumes..
Perimeter Security	100% protection against OWASP Top 10 exploits. - Automated volumetric DDoS mitigation.Block 100% of unauthorized bot/DDoS attacks at the perimeter.	- Automated penetration testing via OWASP ZAP. - Traffic anomaly monitoring via AWS Shield Advanced.	AWS Shield, AWS WAF Web ACLs, and edge AWS API Gateway rate limiters.
Identity & Access	- Rigid FAPI profile enforcement. - Instant (< 500ms) token revocation propagation.	Compliance scanning via OAuth/OIDC Conformance Suites.	Isolated Identity Provider Account (ForgeRock / Ping / Keycloak) and API

		Custom performance testing profiles.	Gateway Custom Authorizers.
API Latency	Public API end-to-end response < 300ms (under 95th percentile peak loads).	- End-to-end load distribution scripts. - Deep-dive tracing metrics inside AWS CloudWatch.	AWS CloudFront caching tiers, AWS API Gateway, and container service pod tiers.
Throughput Capacity	- Sustain baseline 10,000 requests per second (RPS). - Handle a burst capacity of 15,000 RPS.	- Distributed scalability load injection tests. - Autoscaling simulation suites.	Core compute partitions (AWS EKS/ECS) and database connection pools (AWS RDS).
Data Sync Latency	- Cross-region storage latency < 1 second. - Cloud-to-mainframe synchronization loop < 3 seconds.	- Replication lag checking scripts via AWS RDS Metrics. - Queue length tracking inside AWS SQS.	AWS Aurora Global Databases and event-driven data streaming layers (AWS MSK / Kafka).
Data Retention	Maintain audit histories and transactional ledger traces for exactly 7 Years (2,555 days).	- Compliance rule validations within AWS Config. - S3 access policy enforcement verification.	Centralized logging repository with AWS S3 Object Lock running in WORM compliance mode.
System Availability	Platform core infrastructure remains completely responsive during localized zone failures.	- Real-time status reporting via AWS Health Dashboard. - Active availability zone teardown drills.	Multi-Availability Zone architectures spanning all Virtual Private Clouds (VPCs).
Disaster Recovery	- Recovery Point Objective (RPO) < 1 sec. - Recovery Time Objective (RTO) < 15 min.	- Automated, full-scale failover simulations. - Disaster tracking metrics inside AWS Resilience Hub.	Cross-region AWS Aurora Global Databases, multi-region keys, and dynamic AWS Route 53 failover patterns.

Compute Efficiency	• Maintain host CPU consumption between 40% – 70%. • Zero idle server waste.	• Real-time resource sizing checks via AWS Compute Optimizer. • Cluster scaling tracking.	High-efficiency AWS Graviton-powered container node groups inside AWS EKS / ECS.
Storage Optimization	• Reduce live file volumes by automatically offloading inactive metrics every 90 days.	• Storage visualization using S3 Storage Lens. • Lifecycle script execution checking.	AWS S3 Lifecycle Policies configuring automated archiving transitions down to cold AWS S3 Glacier tiers.
Regulatory Audit	• Continuous automated posture checks against global open banking rules (PSD2/FDX/SAMA).	• Unified regulatory reporting dashboards via AWS Security Hub. • Tracking profiles inside AWS Audit Manager.	Baseline compliance configurations handled through AWS Config blueprints across all accounts.
Observability Logs	• 100% log capture across all systems. • Centralized trace assembly < 60 seconds from generation.	• Log querying and pipeline validation via AWS OpenSearch Service. • Error alert triggers.	Log collection pipelines combining AWS CloudTrail, AWS CloudWatch Logs, and AWS X-Ray agents.

Appendix B: Cost Optimization & Attribution Matrix

This appendix provides a structured evaluation framework for linking operational expenses to their true business drivers, ensuring financial accountability and resource efficiency.

Cost Domain	Identified Bleed Point	Architectural Fix / Optimization Strategy	Expected Financial Impact
Compute Infrastructure	 OpEx	• Migrate production compute tiers to AWS Graviton (ARM64) instances. • Implement horizontal pod autoscaling (HPA) driven by AWS CloudWatch custom API transaction metrics rather than standard CPU boundaries.	• 20% – 40% Reduction in direct compute instance expenditures
Data Streaming & Sync	 OpEx	• Re-architect data movement pipelines to prefer local-AZ communication routes. • Consolidate distributed data processing queues into a single shared AWS MSK (Kafka) cluster using cluster partitions instead of isolated multi-account clusters.	• 25% – 35% Decrease in monthly AWS data transfer billing.
Network Security Edge	 OpEx	• Consolidate lower-environment ingress endpoints into a single shared network account using path-based routing rules. • Replace idle staging NAT Gateways with cheaper internal VPC Endpoints where public routing is not strictly needed.	• 40% Savings on development network gateway overhead.
Database Compute	 OpEx	• Transition standard relational clusters to AWS Aurora Serverless v2. • Configure the database capacity boundaries to dynamically scale between a tight base and peak allocation scale based on live API workloads.	• 30% – 50% Drop in overall relational database operational costs.

Long-Term Storage	 OpEx	• Implement automated AWS S3 Lifecycle Policies. • Configure the rules to automatically drop log objects down to cold AWS S3 Glacier Flexible Retrieval storage classes after a strict 90-day retention window.	• 60% – 80% Reduction in historical audit log storage pricing.
Licensing & Software	 OpEx	• Modernise the identity tier by standardising on containerised open-source distributions of Keycloak or negotiating unified multi-tenant pricing for enterprise engines.	• 65% Drop in non-production infrastructure utility costs.
Development Sandboxes	 OpEx	• Deploy automated environment scheduling scripts using AWS Instance Scheduler. • Programmatically scale down all developer sandbox pods and testing database instances to zero outside standard working hours,	• Use AWS RDS Reserved Instances (1 or 3-year commitment) to slice baseline costs by up to 60%.
Cryptographic Hashing	 OpEx	• Implement an edge-level cryptographic cache framework using AWS Lambda@Edge and a distributed AWS ElastiCache ledger. • Validate active token claims at the CDN tier to avoid downstream compute costs.	• 20% Reduction in core application API compute overhead.
Hardware Migration	 CapEx	• Transition on-premises physical backup workloads to a cloud-based hybrid backup strategy. • Move traditional hardware security modules to AWS CloudHSM, switching the infrastructure model to a cloud utility payment scheme	• Significant Reduction in upfront physical hardware lifecycle procurement.

Regulatory Tooling	 CapEx	• Standardise the audit capture system on native automated governance blueprints using AWS Config and AWS Audit Manager templates. • Generate automated, audit-ready reports directly from cloud-native dashboards.	• 35% Reduction in internal compliance engineering hours.
---------------------------	---	---	---

Appendix C: Enterprise Security Risk Register

This register serves as the central repository for identifying, assessing, and tracking security vulnerabilities and threats across the organization, enabling leadership to make informed, risk-based decisions.

Security Vulnerability Description	Likelihood (1-5)	Impact (1-5)	Risk Rating (1-25)	Core Technical Mitigation Strategy
Token Theft & Replay Attacks via TPP Breach An external fintech partner suffers a breach, exposing active customer Access Tokens and Refresh Tokens to malicious actors.	4	5	20 (High)	• Implement strict Financial-Grade API (FAPI 1.0/2.0) security profiles. • Enforce Token Binding using mTLS client certificate thumbprints embedded within access tokens so stolen tokens fail authentication outside the origin network context.
Stale Consent Leakage A user revokes data privileges in the banking app, but distributed caching delay leaves a time window where the TPP can still siphon transactional records.	4	4	16 (High)	• Enforce an instantaneous backchannel event broker loop. • Publish immediate revocation payloads to an in-memory cluster (AWS ElastiCache) checked by AWS API Gateway custom authorizers before executing any API callout.
Volumetric / Layer 7 Denial of Wallet Exploits Malicious actors launch heavy, automated data querying loops through unthrottled public endpoints, intentionally driving up utility processing fees.	4	3	12 (Medium)	• Configure strict consumption tiers and rate-limiting rules directly inside AWS API Gateway. • Map AWS WAF custom rate shields directly to the individual TPP identity parsed at the mTLS edge.
PII Contamination in Central Log Repositories Developers accidentally dump	3	4	12 (Medium)	• Activate AWS CloudWatch Logs Data Protection regex

unencrypted user account balances, legal names, or authentication strings into system error logs during debugging.				patterns across all production logging zones • Deploy AWS Macie over long-term data buckets to automatically redact sensitive matching strings before persistence.
Lateral Movement via Compromised Container Pod An attacker exploits an application vulnerability inside an EKS cluster pod and attempts to pivot into adjacent databases or identity systems.	3	4	12 (Medium)	• Implement strict network segmentation using Kubernetes Network Policies to deny all non-whitelisted cross-pod traffic. • Leverage AWS IAM Roles for Service Accounts (IRSA) to restrict access profiles per container.
Cryptographic Key or Misconfiguration A single broad IAM policy or hardcoded key context exposes master encryption keys, compromising data at rest across databases and filesystems.	2	5	10 (Medium)	• Utilize isolated, account-specific customer-managed keys inside AWS KMS featuring automatic 90-day rotation rules. • Mandate strict cryptographic decryption contexts that match individual service metadata strings.
Certificate Revocation List (CRL) Checking Timeout The external Open Banking trust authority hosting the active list of valid TPP certificates goes offline or times out under heavy traffic loads.	3	3	9 (Medium)	• Build a high-availability background worker task using AWS Lambda to fetch, stream, and cache external CRL maps locally every few minutes inside a fast, internal repository. • Fall back safely to the local copy if the external server times out.
Audit Trail Erasure by Compromised Administrative Role	1	5	5 (Low)	• Route all infrastructure audit records via AWS

An inside actor or rogue administrative identity gains control plane privileges and tries to wipe configuration trails or system access records.				CloudTrail into a separate, isolated logging account. Enforce an unbreakable AWS S3 Object Lock running in WORM compliance mode with a strict 7-year legal block window.
Direct Connect Route Flapping & Partitioning A minor network disruption on the primary physical fiber circuit causes routing loops to rapidly bounce, degrading connection speeds to the core datacenter.	2	2	4 (Low)	- Design a highly resilient active-passive hybrid path structure using explicit Asymmetric BGP Community Weights on the primary Direct Connect loop. - Map an automated AWS Site-to-Site VPN backup line to process fallback packets seamlessly.
Unauthorized Sandbox Exfiltration Fintech developers testing within the mock sandbox environments attempt to query adjacent enterprise subnets or pull mock datasets out of the cloud boundaries.	2	2	4 (Low)	Enforce DNSSEC (Domain Name System Security Extensions) protocols on all AWS Route 53 hosted routing zones to authenticate origin traffic cryptographically.
Third-Party API Supply Chain Poisoning: An external vendor endpoint (e.g., an SMS provider or credit bureau) is compromised, injecting malicious data back into the EKS processing engine.	3	4	12 (Medium)	- Completely decouple the Developer Sandbox Account from the master AWS Transit Gateway loop. - Enforce absolute perimeter boundary monitoring using automated AWS Config rules and restricted outbound NAT gateway rules.

Appendix D: Enterprise RACI Matrix

This matrix defines the clear operational boundaries, decision rights, and delivery expectations across cross-functional teams to eliminate role confusion and accelerate project execution.

Key Architectural Deliverable	Enterprise Architecture Team	Security & Compliance (CISO)	Cloud Engineering (DevOps)	Core Banking Infra	Identity & Access (IAM)	Fintech / API Product (PM)
Perimeter Defense & Ingress (WAF, Shield, mTLS)	<i>A</i>	<i>R</i>	<i>R</i>	<i>C</i>	<i>I</i>	<i>I</i>
Multi-Account Governance & Landing Zone	<i>R</i>	<i>C</i>	<i>A</i>	<i>C</i>	<i>I</i>	<i>I</i>
Core API & Microservice Compute (EKS/ECS)	<i>C</i>	<i>I</i>	<i>A</i>	<i>I</i>	<i>I</i>	<i>R</i>
Financial-Grade Token & Consent Engine	<i>C</i>	<i>A</i>	<i>R</i>	<i>R</i>	<i>I</i>	<i>C</i>
Hybrid Direct Connect & Core Mainframe Sync	<i>C</i>	<i>I</i>	<i>R</i>	<i>I</i>	<i>A</i>	<i>I</i>
WORM Logging, Audit Trails & Compliance Scanning	<i>I</i>	<i>A</i>	<i>R</i>	<i>C</i>	<i>R</i>	<i>I</i>
Disaster Recovery (DR) Execution & Failover	<i>C</i>	<i>C</i>	<i>A</i>	<i>R</i>	<i>R</i>	<i>I</i>
Fintech Onboarding & Sandbox Management (DCR)	<i>I</i>	<i>C</i>	<i>R</i>	<i>R</i>	<i>I</i>	<i>A</i>

Cloud FinOps & Cost Optimization Matrix	<i>C</i>	<i>I</i>	<i>R</i>	<i>I</i>	<i>I</i>	<i>A</i>
Security Risk Register & Incident Response	<i>I</i>	<i>A</i>	<i>R</i>	<i>R</i>	<i>C</i>	<i>C</i>

About the Author

Amit Kulkarni

Senior Systems Architect

📍 Pune, India | ✉️ amitkwork2920@gmail.com | 🔗 [LinkedIn](#) | 💻 [Github](#)

I'm a Technology-agnostic architecture leader having 18+ years of partnership with 15+ clients across BFSI, Retail, Insurance, Healthcare, Travel, Media, and the Public Sector. Proven track record in cloud transformation (AWS, Azure), legacy COTS and mainframe modernization, security posture assessments, microservices modernization, event-driven architectures, and working knowledge on AI initiatives.

Architectural Specialization (Recent Focus)

During a recent deliberate transition period away from formal corporate employment, I dedicated my time to advanced deep-dives into enterprise cloud topologies, edge cases, and failure domains. This case study on *Open Banking Reference Architecture on AWS* is part of a broader body of architectural research aimed at critiquing standard vendor blueprints, identifying hidden system vulnerabilities, and building production-ready engineering remediations for high-throughput transactional environments.

I am currently seeking my next high-impact engineering role where I can solve complex distributed systems challenges and help scale robust, mission-critical infrastructure teams.